

## API — это разговор двух программ

Когда браузер открывает сайт, он отправляет серверу запрос и получает ответ. REST API работает так же, только вместо готовой HTML-страницы обычно передаёт данные в JSON.

В нашем тесте роль клиента играет Java-код. Он говорит серверу: «выполни такую операцию по такому адресу, вот мои данные». Сервер отвечает: «операция закончилась с таким статусом, вот результат».

**Request** — то, что тест отправляет: HTTP-метод, адрес, параметры, заголовки, авторизация и иногда body.

**Response** — то, что сервер вернул: status code, headers и body.

## Из чего состоит HTTP request

| Часть    | Пример         | Смысл                |
|----------|----------------|----------------------|
| Method   | GET            | Прочитать данные     |
| Base URL | https://...com | Какой сервер         |
| Path     | /booking       | Какой endpoint       |
| Headers  | Content-Type   | Служебная информация |
| Body     | {...}          | Отправляемые данные  |

Полный адрес получается из base URL и path. Например: `https://restful-booker.herokuapp.com + /ping`.

## Зачем здесь REST Assured

Java сама умеет отправлять HTTP-запросы, но низкоуровневый код получается длинным. REST Assured — библиотека, которая позволяет описать request понятной цепочкой.

```
Response response = given()
    .spec(BASE_SPEC)
    .when()
    .get("/ping");
```

Это читается сверху вниз:

1. `given()` — начинаем описывать request.
2. `spec(BASE_SPEC)` — подключаем общие настройки: адрес сервера и заголовки.
3. `when()` — настройки закончились, сейчас выполним действие.
4. `get("/ping")` — отправляем GET на endpoint `/ping`.
5. Результат сохраняем в переменную `response`.

## Почему мы сохраняем Response

Response — обычный Java-объект, внутри которого лежит весь ответ сервера. Из него можно получить статус, тип данных и тело ответа.

```
response.statusCode() // 201
response.contentType() // text/plain
response.asString() // Created
```

После получения actual-значений AssertJ сравнивает их с expected:

```
assertThat(response.statusCode())
    .isEqualTo(201);
```

Читается: «убеждаемся, что фактический status равен ожидаемому 201».

## Как это связано с task01

**Задача:** самостоятельно получить Response от GET `/ping`. Assertions ниже уже проверяют три части ответа. Вам не нужно угадывать новые методы: request собирается ровно как в примере выше.

Если переменная остаётся null, assertion падает потому, что ответа ещё нет. Замените только null на вызов REST Assured.

## Главная мысль листа

REST Assured не «проверяет API магически». Он отправляет HTTP request и возвращает объект Response. Дальше тест обычным Java-кодом читает ответ и сравнивает actual с expected.

## Что такое JSON

JSON — текстовый формат передачи данных. Объект записывается в фигурных скобках, массив — в квадратных.

```
{ "bookingid": 17 }
```

```
[  
  { "bookingid": 17 },  
  { "bookingid": 28 }  
]
```

Первый пример — один объект. Второй — массив из двух объектов. Сервер присылает именно текст, но работать в тесте с текстом неудобно.

## DTO — Java-представление JSON

DTO расшифровывается как Data Transfer Object — объект для передачи данных. Мы создаём Java-класс с полями, соответствующими JSON.

```
public record BookingIdResponse(  
    int bookingid) {  
}
```

JSON-поле bookingid попадает в компонент record с тем же именем. После преобразования можно вызвать booking.bookingid() и получить обычное число.

## Кто выполняет преобразование

REST Assured получает body ответа. Jackson читает JSON и создаёт Java-объекты. В проекте Jackson уже подключён, поэтому достаточно вызвать response.as(...).

### JSON-ТЕКСТ → JACKSON → JAVA DTO

Для одного объекта:

```
BookingResponse booking =  
    response.as(BookingResponse.class);
```

BookingResponse.class сообщает Jackson, объект какого типа нужно создать.

## Почему для списка нужен TypeRef

Ответ GET /booking — не один объект, а список. Нам нужен тип List<BookingIdResponse>. Из-за устройства generic-типов Java нельзя написать List<...>.class. REST Assured даёт оболочку TypeRef, которая сохраняет полное описание типа.

```
List<BookingIdResponse> bookings =  
    response.as(new TypeRef<>() {  
    });
```

Тип переменной слева подсказывает компилятору, что внутри TypeRef ожидается именно список BookingIdResponse.

## Как читать проверку списка

```
assertThat(bookings)  
    .isNotEmpty()  
    .allSatisfy(booking ->  
        assertThat(booking.bookingid())  
            .isPositive());
```

1. assertThat(bookings) — проверяем весь список.
2. isNotEmpty() — в нём есть хотя бы один элемент.
3. allSatisfy — запускаем внутреннюю проверку для каждого элемента.
4. Переменная booking внутри лямбды — очередной DTO из списка.
5. bookingid() — getter record, возвращающий ID.

## Как это связано с task02

Сначала получите Response от GET /booking. Затем отдельной строкой преобразуйте body в подготовленную переменную bookings. Assertions уже написаны — ваша задача состоит только из двух понятных действий: получить ответ и превратить JSON-массив в Java-список.

## Почему мы не используем body("\$", matcher)

REST Assured умеет искать значения внутри JSON своим DSL. Но для связанного ответа основной путь курса — DTO. Так вы работаете с типами Java, получаете подсказки IDE и проверяете данные через знакомый AssertJ.

## GET читает, POST создаёт

Для GET обычно достаточно адреса. POST создаёт новый resource, поэтому серверу нужно передать данные будущей записи. Они находятся в request body.

В тесте данные представлены DTO:

```
BookingRequest expected =  
    uniqueBooking("Create");
```

Имя expected важно: это не просто body. Это ожидаемое состояние записи, с которым позже сравнивается ответ сервера.

## Как DTO становится JSON

```
Response response = given()  
    .spec(BASE_SPEC)  
    .contentType(ContentType.JSON)  
    .body(expected)  
    .when()  
    .post("/booking");
```

contentType(JSON) сообщает серверу: «body отправлено в формате JSON». body(expected) передаёт Java DTO. Jackson сериализует его — превращает Java-объект в JSON-текст.

**JAVA DTO → JACKSON → JSON → SERVER**

## Акцепт и Content-Type — не одно и то же

| Заголовок    | Вопрос                                          |
|--------------|-------------------------------------------------|
| Accept       | В каком формате клиент хочет получить response? |
| Content-Type | В каком формате клиент отправляет request body? |

BASE\_SPEC уже содержит Accept JSON. Для POST дополнительно указываем Content-Type JSON, потому что у request появилось body.

## Форма create response

```
{  
  "bookingid": 42,  
  "booking": {  
    "firstname": "Create-...",  
    "lastname": "Student"  
  }  
}
```

Это wrapper: снаружи находятся ID и вложенный booking. Поэтому response DTO повторяет ту же форму:

```
record CreateBookingResponse(  
    int bookingid,  
    BookingResponse booking) {  
}
```

Если структура Java не соответствует JSON, Jackson не сможет корректно заполнить объект.

## Получаем response DTO

```
CreateBookingResponse actual =  
    response.as(CreateBookingResponse.class);
```

Теперь actual.bookingid() даёт созданный ID, а actual.booking() — данные созданной записи.

```
assertThat(actual.booking())  
    .usingRecursiveComparison()  
    .isEqualTo(expected);
```

Recursive comparison проходит по всем полям, включая вложенный bookingdates, и показывает различия.

## Зачем регистрировать ID

Тест создал данные на общем сервере. Их нужно удалить после теста. Поэтому ID сразу добавляется в createdBookingIds. Метод @AfterEach попытается удалить запись даже тогда, когда последующая assertion упадёт.

Правильный порядок: получили ID → сразу зарегистрировали cleanup → продолжили проверки.

## План task03

1. Собрать POST с expected в body.
2. Сохранить результат в response.
3. Преобразовать body в CreateBookingResponse actual.
4. Добавить actual.bookingid() в tracker.
5. Не переписывать готовые assertions снизу.

## Ответ POST ещё ничего не гарантирует

Представим дефект: сервер вернул правильный create response, но не записал данные в базу. Тест, проверивший только POST, будет зелёным. Чтобы проверить реальное состояние системы, выполняют read-back — отдельный GET созданной записи.

**POST → ID → GET ПО ID → SAVED DTO → СРАВНЕНИЕ**

## Path parameter выбирает resource

```
Response response = given()
    .spec(BASE_SPEC)
    .pathParam("id", bookingId)
    .when()
    .get("/booking/{id}");
```

{id} — место в path. pathParam("id", bookingId) подставляет туда фактическое число. Если ID равен 42, сервер получает GET /booking/42.

Это лучше склейки строк: параметр явно назван, REST Assured корректно кодирует значение, а URL остаётся читаемым.

## Проверяем сохранённый DTO

```
assertThat(response.statusCode())
    .isEqualTo(200);

BookingResponse actual =
    response.as(BookingResponse.class);

assertThat(actual)
    .usingRecursiveComparison()
    .isEqualTo(expected);
```

expected появился до POST. actual прочитан с сервера после POST. Их равенство доказывает, что отправленные данные сохранились.

## Query parameter фильтрует список

```
given()
    .spec(BASE_SPEC)
    .queryParams("firstname",
        expected.firstname())
    .queryParams("lastname",
        expected.lastname())
    .when()
    .get("/booking");
```

Здесь endpoint остаётся /booking, но к URL добавляется строка фильтра: ?firstname=...&lastname=.... Ответом всё равно будет массив, поэтому его десериализуют через TypeRef<List<BookingIdResponse>>.

## Path и query отвечают на разные вопросы

|                 |                                             |
|-----------------|---------------------------------------------|
| Path parameter  | «Дай конкретную запись с ID 42»             |
| Query parameter | «Дай список записей, подходящих под фильтр» |

/booking/42 и /booking?firstname=Ivan — разные контракты.

## Почему проверяем свой ID

Restful Booker — общий сервис. Даже при фильтрации в списке могут оказаться чужие записи. Мы не утверждаем, что результат имеет точный размер. Мы проверяем более устойчивый факт: среди найденных ID есть createdId нашего теста.

```
assertThat(actual)
    .extracting(BookingIdResponse::bookingid)
    .contains(createdId);
```

extracting превращает список DTO в список их ID, после чего contains ищет наш ID.

## Tasks04–05

**Task04:** GET конкретной записи через path parameter, DTO и полное сравнение.

**Task05:** GET списка через query parameters, TypeRef и поиск своего ID.

## Негативный сценарий — ожидаемая ошибка

Если запросить несуществующую запись, корректным результатом является 404 Not Found. Для теста это не авария: мы специально проверяем контракт ошибки.

```
Response response = given()
    .spec(BASE_SPEC)
    .pathParam("id", unknownId)
    .when()
    .get("/booking/{id}");

assertThat(response.statusCode())
    .isEqualTo(404);
```

Тест зелёный, если сервер правильно отказал. Успех теста не всегда означает HTTP 200.

## Почему нужны логи

Когда тест падает, одного сообщения «expected 200, actual 500» мало. Нужно увидеть, куда ушёл request и что вернул сервер. Но `log().all()` во всех тестах создаёт тысячи строк и может показать token или password.

Практичный вариант — логировать только при провале validation:

```
given()
    .log().ifValidationFails()
    ...
response.then()
    .log().ifValidationFails();
```

Для общих failure logs можно подключить `ErrorLoggingFilter`.

## Что искать в логе

1. Правильный base URL и path.
2. Фактический HTTP method.
3. Path/query parameters.
4. Content-Type и безопасные headers.
5. Request body соответствует DTO.
6. Status, response headers и body.

Так ошибка классифицируется: неверный тест, некорректные данные, дефект продукта или проблема окружения.

## Зачем RequestSpecification

Base URL и Асепт одинаковы почти у всех запросов. Если повторять их в каждом тесте, важное действие теряется среди технических строк.

```
RequestSpecification BASE_SPEC =
    new RequestSpecBuilder()
        .setBaseUrl(BASE_URL)
        .setAccept(ContentType.JSON)
        .build();
```

Builder по шагам создаёт объект настроек. Метод `build()` завершает сборку.

## Как specification применяется

```
given()
    .spec(BASE_SPEC)
    .pathParam("id", bookingId)
    .when()
    .get("/booking/{id}");
```

Сначала подключается общее, потом добавляется специфичное для request. Specification не отправляет запрос сама — она только хранит настройки.

## Что нельзя прятать в specification

- Body и expected конкретного теста.
- ID, принадлежащий одному сценарию.
- Business assertions.
- Ожидаемый status, если happy и negative cases различаются.

Цель specification — убрать повторяющуюся HTTP-механику, а не спрятать смысл теста.

## Tasks06–07

**Task06:** на request включить `log().ifValidationFails()`, сохранить `Response`, а затем вызвать `response.then().log().ifValidationFails().statusCode(404)`. Именно REST Assured validation связывает ожидаемый status с условным логированием.

**Task07:** самостоятельно собрать `RequestSpecification` через builder и применить её к реальному GET.

## Authentication и authorization

**Authentication** отвечает на вопрос «кто вы?». Клиент отправляет username/password и получает token.

**Authorization** отвечает на вопрос «можно ли вам выполнить действие?». Token прикладывается к защищённым PUT, PATCH и DELETE.

Это два разных шага, даже если в разговоре оба называют «авторизацией».

## Credentials — обычный request DTO

```
AuthRequest credentials =
    new AuthRequest("admin", "password123");

Response response = given()
    .spec(BASE_SPEC)
    .contentType(ContentType.JSON)
    .body(credentials)
    .when()
    .post("/auth");
```

Здесь нет специальной магии безопасности: это обычный POST с JSON body.

## Token — обычный response DTO

```
{ "token": "abc123..." }
```

```
record AuthResponse(String token) {}
```

```
AuthResponse actual =
    response.as(AuthResponse.class);

assertThat(actual.token()).isNotBlank();
```

Затем token передаётся защищённому endpoint в cookie:

```
.cookie("token", actual.token())
```

## Что проверяет negative auth

Happy path доказывает, что правильные credentials дают token. Но он не доказывает, что неправильные credentials отклоняются. Поэтому нужны случаи:

- правильный user + неправильный password;
- неправильный user + правильный password;
- оба значения неправильные.

Для каждого сервер возвращает одинаковую форму:

```
{ "reason": "Bad credentials" }
```

Она десериализуется в AuthErrorResponse.

## Зачем @ParameterizedTest

Логика request и assertions одинакова, меняются только входные значения. Вместо трёх почти одинаковых методов JUnit запускает один метод три раза.

```
@CsvSource({
    "admin, wrong-password",
    "wrong-user, password123",
    "wrong-user, wrong-password"
})
```

При каждом запуске значения очередной строки попадают в параметры username и password.

## Когда параметризация не нужна

Если у сценариев разные действия или разные ожидания, не пытайтесь записать их в одну таблицу с множеством условий. Parameterized test подходит для одного контракта и набора однотипных inputs.

## Tasks08–09

**Task08:** POST credentials → response → AuthResponse → непустой token.

**Task09:** взять username/password из JUnit-параметров → POST → AuthErrorResponse → проверить reason.

## Общая схема mutation-теста

PUT и PATCH изменяют данные. Их нельзя считать проверенными только потому, что endpoint вернул успешный status. Нужен снимок состояния до действия и read-back после.

**ORIGINAL → ACTION → RESPONSE → GET → SAVED → ASSERTIONS**

## PUT — полная замена

В PUT передаётся полный объект updated. Он должен заменить текущее состояние resource.

```
Response updateResponse = given()
    .spec(BASE_SPEC)
    .cookie("token", getToken())
    .contentType(ContentType.JSON)
    .pathParam("id", bookingId)
    .body(updated)
    .when()
    .put("/booking/{id}");
```

Каждая часть уже знакома: общая spec, token, формат body, ID, DTO и HTTP method.

## Что проверяем после PUT

1. Update response имеет ожидаемый status.
2. Отдельный GET читает resource.
3. JSON превращается в BookingResponse saved.
4. saved полностью совпадает с updated.

Именно последняя проверка доказывает, что сервер сохранил все поля.

## PATCH — меняем только часть

Для PATCH часто не нужен полный DTO: отправляется маленький JSON только с изменяемыми полями. Здесь Map практична и хорошо показывает намерение.

```
Map<String, Object> patch =
    Map.of("lastname", changedLastName);
```

Jackson превратит Map в:

```
{ "lastname": "Patched-..." }
```

Request собирается как PUT, но method — patch, а body — patch.

## Changed и preserved fields

После PATCH недостаточно проверить новый lastname. Сервер мог случайно обнулить остальные поля. Поэтому GET после PATCH проверяет две группы:

|           |                                        |
|-----------|----------------------------------------|
| Changed   | lastname равен новому значению         |
| Preserved | firstname, price, dates равны original |

## Почему здесь SoftAssertions

Поля описывают одно сохранённое состояние. Если повреждено несколько, полезно увидеть их все за один запуск.

```
SoftAssertions.assertSoftly(softly -> {
    softly.assertThat(saved.lastname())
        .isEqualTo(changedLastName);
    softly.assertThat(saved.firstname())
        .isEqualTo(original.firstname());
});
```

softly — объект-сборщик проверок. Он запоминает ошибки внутри лямбды и сообщает их вместе после её завершения.

## Tasks10–11

**Task10:** PUT полного DTO и полное сравнение read-back.

**Task11:** PATCH маленькой Map и проверка changed/preserved state.



## Отказ должен оставлять данные неизменными

Task12 отправляет PUT без token. Ожидаемый status 403 доказывает, что endpoint отклонил request. Но этого мало: дефектный сервер мог вернуть 403 и всё равно применить update.

Поэтому negative mutation имеет два assertions:

1. `deniedResponse.statusCode() == 403;`
2. GET после отказа возвращает исходный `original`.

**ORIGINAL → PUT БЕЗ TOKEN → 403 → GET → ORIGINAL**

## Почему `forbiddenUpdate` отличается от `original`

Если отправить те же данные, что уже сохранены, нельзя понять, применил сервер запрещённый request или нет. Поэтому `forbiddenUpdate` намеренно содержит другие значения. Тогда read-back однозначно показывает результат.

Test data выбираются так, чтобы различать возможные исходы сценария.

## DELETE — тоже mutation

DELETE меняет state: resource должен перестать существовать. Проверка только status 201 говорит лишь о response операции.

```
Response deleteResponse = given()
    .spec(BASE_SPEC)
    .cookie("token", getToken())
    .pathParam("id", bookingId)
    .when()
    .delete("/booking/{id}");
```

После этого отдельный GET должен вернуть 404.

## Tracker и ручное удаление

При создании ID был добавлен в `createdBookingIds`. Если основной сценарий сам успешно удалил resource, ID нужно убрать из tracker:

```
createdBookingIds.remove(bookingId);
```

Иначе `@AfterEach` попытается удалить его повторно. Это не всегда опасно, но создаёт лишний запрос и шум.

## Как работает `@AfterEach`

```
@AfterEach
void removeCreatedBookings() {
    for (int id : createdBookingIds) {
        deleteBookingIfPresent(id);
    }
    createdBookingIds.clear();
}
```

JUnit вызывает этот метод после каждого теста — и после успешного, и после упавшего. Поэтому cleanup не зависит от того, дошёл основной сценарий до конца или нет.

## Cleanup не должен скрывать главную ошибку

Метод `deleteBookingIfPresent` ловит ошибку очистки и пишет сообщение, а не заменяет ею исходное падение assertion. Иначе вместо настоящего дефекта вы увидите только «cleanup failed».

### Tasks12-13

**Task12:** запрещённый PUT → 403 → GET → original не изменился.

**Task13:** DELETE с token → удалить ID из tracker → GET 404.



## Почему тесты начинают падать случайно

API-тесты часто становятся flaky не из-за REST Assured, а из-за общих данных. Один тест создаёт static ID, второй его обновляет, третий удаляет. При другом порядке или параллельном запуске всё ломается.

Независимый тест сам создаёт нужное состояние и не использует результат другого тестового метода.

## Признаки независимости

- Request DTO создаётся внутри теста.
- Данные уникальны, например через UUID.
- ID хранится в локальной переменной.
- Expected не находится в static mutable field.
- Cleanup удаляет только созданные этим тестом resources.
- Метод проходит отдельно и в полном suite.

## Зачем task14 создаёт две записи

Один успешный create/read ещё не показывает, что тест не перепутывает данные. В task14 создаются first и second с разными expected DTO. Затем каждый ID читается отдельно и сравнивается со своим expected.

```
firstId → firstActual → firstExpected  
secondId → secondActual → secondExpected
```

Дополнительно ID должны быть положительными и различаться.

## Финальный сценарий task15

1. Создать unique initial.
2. POST, проверить status, получить CreateBookingResponse.
3. Сохранить ID в подготовленную переменную bookingId.
4. GET по ID, получить DTO, сравнить с initial.
5. Получить token.
6. PATCH additionalneeds.
7. GET, проверить изменённое и сохранённое.
8. DELETE с token.
9. GET должен вернуть 404.
10. finally страхует cleanup при раннем падении.

## Зачем try/finally, если есть @AfterEach

В финальном задании cleanup уже принадлежит finally. После POST сохраните ID в переменную bookingId, но не добавляйте его в общий tracker: иначе запись будут пытаться удалить дважды. Если ID успели получить, cleanup выполнится независимо от места падения.

```
Integer bookingId = null;  
try {  
    // основной CRUD-flow  
} finally {  
    if (bookingId != null) {  
        deleteBookingIfPresent(bookingId);  
    }  
}
```

В реальном проекте обычно выбирают один согласованный механизм cleanup, чтобы не дублировать действия.

## Финальный тест не должен быть единственным

Большой CRUD-flow показывает, что части работают вместе, но при падении сложнее локализовать причину. Короткие тесты на create, auth, update и delete всё ещё нужны. Финальный flow дополняет их, а не заменяет.

## Что значит «задача решена»

Тест проходит несколько раз подряд, отдельно и вместе с классом; после запуска не остаются данные; по сообщению об ошибке понятно, на каком шаге и какие expected/actual разошлись.

## Роли инструментов

| Инструмент   | Ответственность                                                         |
|--------------|-------------------------------------------------------------------------|
| JUnit        | Находит и запускает тесты, вызывает Before/After, подставляет параметры |
| REST Assured | Собирает HTTP request и возвращает Response                             |
| Jackson      | Сериализует DTO в JSON и десериализует JSON в DTO                       |
| AssertJ      | Сравнивает actual Java-данные с expected                                |
| DTO          | Типизированное Java-представление request/response JSON                 |

## Алгоритм нового endpoint

1. Прочитать документацию: method, path, params, auth, request и response.
2. Определить, что является главным риском сценария.
3. Создать request DTO с уникальными данными.
4. Собрать request и сохранить Response.
5. Проверить точный status и важный content type.
6. Преобразовать body в response DTO.
7. Сравнить response DTO с request/expected.
8. После mutation прочитать state отдельным GET.
9. Добавить значимый negative case.
10. Настроить cleanup и failure logging.
11. Запустить отдельно, в suite и повторно.

## Мини-словарь методов

|                           |                                    |
|---------------------------|------------------------------------|
| given()                   | Начать описание request            |
| spec(x)                   | Подключить общие настройки         |
| contentType(JSON)         | Request body отправляется как JSON |
| body(dto)                 | Передать объект для сериализации   |
| pathParam                 | Подставить часть path              |
| queryParam                | Добавить фильтр URL                |
| cookie                    | Передать token Restful Booker      |
| get/post/put/patch/delete | Выполнить HTTP method              |
| response.as(Class)        | JSON-object → DTO                  |
| response.as(TypeRef)      | JSON-array → generic collection    |
| response.asString()       | Получить body как текст            |

## Структура проекта после обучения

| Папка/слой   | Что там                                           |
|--------------|---------------------------------------------------|
| tests        | Сценарии, expected и assertions                   |
| client       | Методы вызова endpoint, возвращающие Response/DTO |
| models       | Request и response DTO                            |
| fixtures     | Создание валидных unique test data                |
| config/specs | Base URL, headers, auth/config и logging          |

Не создавайте все слои заранее. Сначала напишите несколько понятных тестов, затем вынесите повторяющуюся механику.

## Если тест упал

1. Request ушёл на правильный URL и method?
2. Body действительно сериализовалось в ожидаемый JSON?
3. Token присутствует и не просрочен?
4. Status говорит об ошибке теста, продукта или окружения?
5. Response body соответствует DTO?
6. Expected построен из тех же данных, что request?
7. Проверяется собственный ID?
8. Mutation подтверждена GET?

## Что нужно запомнить

Не требуется помнить каждую функцию REST Assured. Нужно понимать путь данных: Java DTO превращается в JSON request; сервер возвращает JSON response; Jackson создаёт response DTO; AssertJ сравнивает actual с expected; GET после изменения подтверждает state; JUnit гарантирует lifecycle и cleanup.

## Формула рабочего API-теста

**Контракт → собственные данные → request → status → response DTO → business assertions → read-back → negative case → cleanup → диагностика.**